

CHAPITRE 2

GÉNÉRATION DYNAMIQUE DES DOCUMENTS EN PHP

Plan

- Partie 1 : Les structures de base en PHP
- Partie 2 : Les types de données structurés en PHP: Les tableaux et les chaînes de caractères
- Partie 3 : Les fonctions en PHP

Partie 1

Les Structures de base en PHP

Plan

- Présentation de PHP
- Variables, constantes et types de données
- Les structures de contrôle
- L'affichage des données et les inclusions

A. Introduction à la technologie PHP

I. Présentation

PHP est un langage de script qui s'inclut dans le langage HTML. La syntaxe du langage PHP provient du C, du Java et du Perl, avec un petit nombre de fonctions inédites par rapport à ces langages. Le but du langage PHP est de permettre aux développeurs de site web d'écrire rapidement des pages web dynamiques.

PHP est un langage interprété (un langage de script) exécuté du côté serveur et non du côté client (un script écrit en Javascript ou une applet Java s'exécute sur votre ordinateur...). La syntaxe du langage provient de celles du langage C, du Perl et de Java. Ses principaux atouts sont:

- La gratuité et la disponibilité du code source

- La possibilité d'inclure le script PHP au sein d'une page HTML;
- La simplicité d'interfaçage avec des bases de données (de nombreux SGBD sont supportés, mais le plus utilisé avec ce langage est MySQL, un SGBD gratuit disponible sur les plateformes Unix, Linux, et Windows)) ;
- L'intégration au sein de nombreux serveurs web (Apache, Microsoft IIS, etc.).

II. Syntaxe de base :

Pour que le script soit interprété par le serveur deux conditions sont nécessaires:

- Le fichier contenant le code doit avoir l'extension **.php** et non .html
- Le code PHP contenu dans le code HTML doit être délimité par les balises

<?php et ?>

La syntaxe du php est très proche de celle du langage C:

- ❑ ; à la fin de chaque ligne d'instructions.
- ❑ {...} pour encadrer un bloc d'instructions
- ❑ les opérateurs de comparaison, logiques et d'affectation sont les mêmes (&&, ||, ==, =...)
- ❑ les symboles des commentaires // pour une seule ligne et /* ... */ pour plusieurs lignes.
- ❑ toute une série de mots réservés qui correspondent à des mots-clés du langage.
- ❑ Des fonctions et des variables prédéfinies du langage

B. Variables, constantes et types de données

I. Utilisation des constantes :

1. Définition d'une constante :

Pour définir une constante, on fait appel à la fonction **define()**.

Exemple 1 :

```
define("SOCIETE","Société internationale d'Import Export" );
echo ("Je travaille à la société : " . SOCIETE) ;
```

2. Constantes prédéfinies :

PHP possède plusieurs constantes prédéfinies. Parmi ces constantes on peut citer :

- **TRUE** = 1
- **FAUX** = 0 ou chaîne vide

- `E_ERROR` = Dénote une erreur autre qu'une "parsing error" (erreur d'analyse) qu'il n'est pas possible de corriger.

II. Utilisation des variables :

En PHP, les variables sont représentées par un signe dollar suivi du nom de la variable. Une variable est automatiquement déclarée dès que vous lui affectez une valeur. L'affectation est réalisée à l'aide de l'opérateur (=).

Exemple :

```
$nom = "Mejdi"; $prix = 15,200 ;
```

PHP tient compte de la casse des caractères dans les noms de variables.

PHP supporte les caractères accentués dans les noms de variable.

1. Détermination de types :

La détermination de types permet de modifier explicitement le type de données d'une variable.

Exemple :

```
$var = 15,6 ; // $var est un double
```

```
$var = (int) $var ; // c'est maintenant un entier (valeur 15)
```

```
$var = (double) $var ; // c'est de nouveau un double (valeur 15,0)
```

```
$var_chaine = (string)$var ; // $var_chaine est une chaîne (valeur "15")
```

2. Les variables dynamiques :

Il est pratique d'avoir parfois des noms de variables qui sont variables. C'est-à-dire un nom de variable qui est affecté et utilisé dynamiquement. Une variable classique est affectée avec l'instruction suivante : `$a = "hello";`

Une variable dynamique prend la valeur d'une variable et l'utilise comme nom d'une autre variable. Dans l'exemple ci-dessous, `hello`, peut être utilisé comme le nom d'une variable en utilisant le `$$` précédent la variable, c'est-à-dire :

```
$$a = "world";
```

A ce niveau, deux variables ont été définies et stockées dans l'arbre des symboles PHP: `$a` avec comme valeur "hello" et `$hello` avec comme valeur "world". Alors, l'instruction :

```
echo "$a $$a";
```

produira le même affichage que :

```
echo "$a $hello";
```

c'est-à-dire : hello world.

3. Fonctions utiles pour les variables :

PHP possède plusieurs fonctions internes permettant de travailler avec les variables :

- string **gettype** (mixed *var*)

Retourne le type de la variable PHP *var*.

Les chaînes de caractères que peut retourner la fonction sont les suivantes :

"integer", "double", "string", "array", "object", "class", "unknown type" (type inconnu)

Exemple :

```
if(gettype($saisie_utilisateur) == "integer")
{
    $age=$saisie_utilisateur ;
}
```

Une autre alternative pour connaître le type d'une variable est l'utilisation de l'une des fonctions suivantes : `is_long()`, `is_double()`, `is_string()`, `is_array()` et `is_object()`.

- int **settype** (string *var*, string *type*)

Définit de façon explicite le type (*type*) de la variable *var*.

Le *type* peut être : integer, double, string, array ou object. La fonction `settype` renvoie TRUE en cas de succès, FALSE sinon.

- int **isset** (mixed *var*)

Sert à déterminer si une variable possède une valeur. Renvoie TRUE si la variable *var* est définie, FALSE sinon.

Exemple :

```
$nom="Ali" ;
echo(isset($nom)); //TRUE
```

- int **unset** (mixed *var*) Sert à détruire une variable.

Exemple :

```
$nom = "Ali" ;
unset($nom) ;
if (isset($nom))
    { echo("Ceci NE s'affiche PAS") ; }
```

- int **empty** (mixed *var*)

Elle renvoie TRUE si la variable n'est pas définie, ou possède une valeur de 0 ou une chaîne vide. Sinon elle renvoie FALSE.

Exemple :

```
$var= 0;
if (empty($var)) { //retourne TRUE
    print 'soit $var vaut 0, soit il n'est pas défini'; }
```

4. La portée des variables

Dépendamment du lieu de déclaration de la variable, il est possible que celle-ci ne soit pas accessible à certains endroits.

C'est notamment le cas des variables déclarées à l'intérieur des fonctions.

Exemple

<?php

```
$variable = 'Salut';
// Déclaration de la fonction
function Ma_Fonction() {
    $variable = 'Bonjour';}
// Appel de la fonction
Ma_Fonction();
echo $variable;
// Affichera "Salut" et non "Bonjour"
```

?>

a. **Variable super-globale \$GLOBALS:**

\$GLOBALS est une variable super-globale référençant toutes les variables disponibles lors de l'exécution du code. Cette variable tableau est donc accessible en tout temps et vous permet d'accéder à vos variables peu importe où vous vous trouvez.

Exemple

<?php

```
$variable = 'Salut';
function Ma_Fonction() {
    $GLOBALS['variable'] = 'Bonjour';
}
Ma_Fonction();
echo $variable;
//Affichera "Bonjour"
```

?>

III. Les types de données

Il n'est pas nécessaire de déclarer le type des données que l'on manipule, celui-ci est affecté dynamiquement par PHP.

En revanche, il n'est pas inutile de connaître les différents types possibles :

1. Booléens

Un booléen (cf les expressions) exprime une valeur VRAI 'TRUE', ou FAUX 'FALSE'. Les expressions suivantes étant strictement identiques, on ne sera pas, à priori, amené à utiliser explicitement un type booléen :

```
<?php
    ($a == 0) == TRUE;
    ($a == 0) != FALSE;
    $a == 0;
?>
```

2. Entiers

Un entier (integer, ou int) est un nombre de l'ensemble des entiers naturels {..., -2, -1, 0, 1, 2, ...}. On peut l'exprimer de trois manières :

```
<?php
    $a = 1234; /* nombre entier en base décimale */
    $a = 02322; /* le même en base octale, préfixé 0 */
    $a = 0x4D2; /* le même en base hexadécimale, préfixé 0x */
?>
```

3. Réels

Un nombre réel (float, ou double), appelé aussi "à virgule flottante", peut être exprimé de deux manières :

```
<?php
    $a = 1.234;
    $a = 1.2e3; /* notation scientifique */
?>
```

4. Chaînes de Caractères

Une chaîne (ou séquence) de caractères doit être encadrée de guillemets :

- ☐ simples (' ') si elle ne contient rien qui nécessite d'être échappé
- ☐ doubles (" ") si l'on veut que les séquences d'échappement et les variables qu'elle contient soient interprétées (on parle de "substitution de variables")

```
<?php
    $version = '4.4.2';
    echo "PHP version $version !"
```

```
?>
```

IV. Les opérateurs PHP

1. Les opérateurs de calcul

Les opérateurs de calcul permettent de modifier mathématiquement la valeur d'une variable

Opérateur	Dénomination	Effet	Exemple	Résultat (pour $x=7$)
+	Opérateur d'addition	Ajoute deux valeurs	$\$x+3$	10
-	Opérateur de soustraction	Soustrait deux valeurs	$\$x-3$	4
*	Opérateur de multiplication	Multiplie deux valeurs	$\$x*3$	21
/	opérateur de division	Divise deux valeurs	$\$x/3$	2.3333333
=	opérateur d'affectation	Affecte une valeur à une variable	$\$x=3$	Met la valeur 3 dans la variable $\$x$
%	opérateur modulo	Donne le reste de la division entière entre 2 nombres	$\$x\%3$	1

2. Les opérateurs d'assignation

Ces opérateurs permettent de simplifier des opérations telles qu'ajouter une valeur dans une variable et stocker le résultat dans la variable. Une telle opération s'écrit habituellement de la façon suivante par exemple : $\$x=\$x+2$. Avec les opérateurs d'assignation il est possible d'écrire cette opération sous la forme suivante : $\$x+=2$. Les autres opérateurs du même type sont les suivants :

Opérateur	Effet
$+=$	addition deux valeurs et stocke le résultat dans la variable (à gauche)
$-=$	soustrait deux valeurs et stocke le résultat dans la variable
$*=$	multiplie deux valeurs et stocke le résultat dans la variable
$/=$	divise deux valeurs et stocke le résultat dans la variable
$\%=$	Donne le reste de la division deux valeurs et stocke le résultat dans la variable
$ =$	Effectue un OU logique entre deux valeurs et stocke le résultat dans la variable
$\wedge=$	Effectue un OU exclusif entre deux valeurs et stocke le résultat dans la variable
$\&=$	Effectue un Et logique entre deux valeurs et stocke le résultat dans la variable
$.=$	Concatène deux chaînes et stocke le résultat dans la variable

3. Les opérateurs d'incrémentation

Ce type d'opérateur permet de facilement augmenter ou diminuer d'une unité une variable. Ces opérateurs sont très utiles pour des structures telles que des boucles, qui ont besoin d'un compteur (variable qui augmente de un en un). Un opérateur de type $\$x++$ permet de remplacer des notations lourdes telles que $\$x=\$x+1$ ou bien $\$x+=1$

Opérateur	Dénomination	Effet	Syntaxe	Résultat (avec x valant 7)
++	Incrémentatio	Augmente d'une unité la variable	$\$x++$	8
--	Décrémentatio	Diminue d'une unité la variable	$\$x--$	6

4. Les opérateurs de comparaison

Opérateur	Dénomination	Effet	Exemple	Résultat
==	opérateur d'égalité	Compare deux valeurs et vérifie leur égalité	$\$x==3$	Retourne 1 si $\$x$ est égal à 3, sinon 0
<	opérateur d'infériorité stricte	Vérifie qu'une variable est strictement inférieure à une valeur	$\$x<3$	Retourne 1 si $\$x$ est inférieur à 3, sinon 0
<=	opérateur d'infériorité	Vérifie qu'une variable est inférieure ou égale à une valeur	$\$x<=3$	Retourne 1 si $\$x$ est inférieur ou égale à 3, sinon 0
>	opérateur de supériorité stricte	Vérifie qu'une variable est strictement supérieure à une valeur	$\$x>3$	Retourne 1 si $\$x$ est supérieur à 3, sinon 0
>=	opérateur de supériorité	Vérifie qu'une variable est supérieure ou égale à une valeur	$\$x>=3$	Retourne 1 si $\$x$ est supérieur ou égal à 3, sinon 0

!=	opérateur de différence	Vérifie qu'une variable est différente d'une valeur	\$x!=3	Retourne 1 si \$x est différent de 3, sinon 0
----	-------------------------	---	--------	---

5. Les opérateurs logiques (booléens)

Ce type d'opérateurs permet de vérifier si plusieurs conditions sont vraies:

Opérateur	Dénomination	Effet	Syntaxe
ou OR	OU logique	Vérifie qu'une des conditions est réalisée	((condition1) (condition2))
&& ou AND	ET logique	Vérifie que toutes les conditions sont réalisées	((condition1) && (condition2))
XOR	OU exclusif	Vérifie qu'une et une seule des conditions est réalisée	((condition1) XOR (condition2))
!	NON logique	Inverse l'état d'une variable booléenne (retourne la valeur 1 si la variable vaut 0, 0 si elle vaut 1)	(!condition)

6. Autres opérateurs

Opérateur	Dénomination	Effet	Syntaxe	Résultat
.	Concaténation	Joint deux chaînes bout à bout	"Bonjour"."Au revoir"	"BonjourAu revoir"
\$	Référencement de variable	Permet de définir une variable	\$MaVariable = 2;	
->	Propriété d'un objet	Permet d'accéder aux données membres d'une classe	\$MonObjet->Propriete	
=>	Associe une valeur à une clé dans un tableau			

L'opérateur	Opérateur de test	Syntaxe	est
? :		[test logique] [expression si vrai] : [expression si faux]	<?php ? \$a= \$b =1; (\$a == \$b) ? \$c= 10 : \$c = 20; // effectue \$c = 10; ?>

C. Les structures de contrôle

I. Les structures conditionnelles :

Les structures conditionnelles permettent de définir des blocs de code qui ne seront exécutés que si certaines conditions sont remplies.

1. Instruction If :

```
if (condition){  
    expression vrai  
    } else {  
    expression faux  
    }
```

PHP dispose également du mot clé elseif : Comme l'expression else, elle permet d'exécuter une instruction après un if dans le cas où le "premier" if est évalué comme FALSE. Mais, à la différence de l'expression else, il n'exécutera l'instruction que si l'expression conditionnelle elseif est évaluée à TRUE.

Une instruction if peut comporter un nombre quelconque d'instructions elseif. La branche finale else permet alors de spécifier le code devant être exécuté si aucune des conditions précédentes n'est remplie.

Exemple :

```
<? php  
$a = 10; $b = 11;  
print "a est égale à $a, b est égale à $b. ";  
if ($ a > $b) { print "a est plus grand que b"; }  
elseif ($ a == $b) { print "a est égal à b"; }  
else {print "==> a est inférieure à b."; }  
?>
```

PHP propose également une autre syntaxe pour l'instruction if : if : ..endif ;

L'exemple suivant illustre cette alternative.

Exemple :

<? php

```
$a = 10; $b = 11;
```

```
print "a est égale à $a, b est égale à $b. ";
```

```
if ($a > $b) : print "a est plus grand que b"; elseif ($a == $b) : print "a est égal à b"; else :
```

```
print "=> a est inférieure à b."; endif ;
```

?>

Dans l'exemple, les accolades ne sont pas utilisées et les conditions testées sont suivies de deux points (:).

La dernière endif sert à signaler la fin du bloc if, remplaçant une accolade fermante}.

2. Les structures de branchement :

```
switch (expression) {  
case resultat1 :  
# instructions à exécuter si l'expression vaut resultat1...  
break ;  
case resultat2 :  
# instructions à exécuter si l'expression vaut resultat2...  
break ;  
... default  
:  
# instructions à exécuter en dernier recours...  
}
```

L'instruction switch équivaut à une série d'instructions if..elseif..elseif....else ; Elle est utilisée pour comparer la même variable (ou expression) avec un grand nombre de valeurs différentes et d'exécuter différentes parties de code suivant la valeur à laquelle elle est égale. L'instruction break a pour but de stopper l'exécution des instructions d'une clause case donnée

Exemple:

```
Switch ($type) {
```

```
case "Femme" : echo "Bonjour Madame" ; break ;
```

```
case "Homme" : echo "Bonjour Monsieur" ; break ;
```

```
default : echo "Erreur !" ;  
}
```

II. Les structures itératives

Les structures itératives (boucles) constituent un moyen d'exécuter un bloc de code un nombre de fois donné, où jusqu'à ce qu'une condition particulière soit satisfaite.

PHP possède deux types de boucles :

- Une boucle **while** : Teste une condition avant ou après chaque itération et ne parcourt de nouveau la boucle que si la condition est vérifiée.
- Une boucle **for** : Le nombre d'itérations est fixé une fois pour toutes, avant l'exécution de la première boucle.

1. La boucle While:

```
while (condition)  
{ //instructions }
```

La signification d'une boucle **while** est très simple. Le PHP exécute l'instruction tant que l'expression de la boucle **while** est évaluée comme **TRUE**. La valeur de l'expression est vérifiée à chaque début de boucle, et, si la valeur change durant l'exécution de l'instruction, l'exécution ne s'arrêtera qu'à la fin de l'itération (chaque fois que le PHP exécute l'instruction, on appelle cela une itération).

Si l'expression du **while** est **FALSE** avant la première itération, l'instruction ne sera jamais exécutée.

Exemple :

```
<? php  
/* Afficher tous les nombres de 1 à 10 */  
$i = 1;  
while ($i <= 10)  
{ print $i++; /*valeur affichée est $i avant l'incrementation (post-incrementation) */ }  
?>
```

Comme l'instruction **if**, **while** accepte une autre syntaxe en utilisant les deux points (:)

Exemple :

```
<?php  
/* Afficher tous les nombres de 1 à 10 : Autrement*/  
$i = 1;  
while ($i <= 10):  
    print $i;  
    $i++;  
endwhile;
```

?>

2. La boucle *do.. While* :

Les instructions *do..while* sont similaires aux instructions *while*, si ce n'est que la condition est vérifiée à la fin de chaque itération, et non au début. Cela signifie que la boucle s'exécute toujours au moins une fois (l'expression n'est testée qu'à la fin de l'itération), ce qui n'est pas le cas lorsque vous utilisez une boucle *while* (l'expression est vérifiée au début de chaque itération).

Exemple :

```
$i = 0;  
do { print $i;  
  } while ($i>0);
```

La boucle ci-dessus ne va être exécutée qu'une seule fois, car lorsque l'expression est évaluée, elle vaut FALSE (car la variable \$i n'est pas plus grande que 0) et l'exécution de la boucle s'arrête.

3. La boucle *For* :

```
for (expr1; expr2; expr3) { instructions  
}
```

L'instruction accepte trois expressions :

- ❑ La première expression (expr1) est évaluée (exécutée) quoi qu'il arrive au début de la boucle.
- ❑ Au début de chaque itération, l'expression expr2 est évaluée. Si l'évaluation vaut TRUE, la boucle continue et l'instruction est exécutée. Si l'évaluation vaut FALSE, l'exécution de la boucle s'arrête.
- ❑ A la fin de chaque itération, l'expression expr3 est exécutée.

L'expression expr2 compare en général la variable de contrôle de boucle à une valeur prédéfinie, mais cela n'a rien d'obligatoire.

Exemple :

```
for ($i=0 ; $i<10 ; $i++)  
{  
  echo  
  "$i"; }  
}
```

PHP supporte aussi la syntaxe alternative suivante pour les boucles *for* :

```
for (expr1; expr2; expr3):  
  instructions; ...; endfor;
```

Exemple:

```
for($i=0 ; $i<10 ; $i++) : echo "$i"; endfor ;
```

4. La boucle foreach():

foreach() permet d'exécuter une série d'instructions pour chacun des éléments d'un tableau.

Prenez note que la boucle réinitialise la position du pointeur interne du tableau mais n'affecte pas le tableau en lui-même. Il travaille en fait sur une copie de celui-ci.

Toute modification faite aux valeurs récupérées ne se répercutera pas sur le tableau en question.

Syntaxes

```
<?php
foreach (tableau as valeur) commandes

foreach (tableau as clef => valeur) commandes

?>
```

La première syntaxe parcourt tous les éléments du tableau et assigne à la variable valeur la valeur de l'élément en cours.

La deuxième syntaxe parcourt tous les éléments du tableau et assigne, en plus, à la variable clef l'index de l'élément en cours.

Exemple

```
<?php
$tableau = array(
    'Nom' => 'Xenon_54',
    'Ville' => 'Inconnue'
)
foreach ($tableau as $clef => $valeur)
{ echo $clef . ' : ' . $valeur . '<br />'; }
?>
```

5. BREAK et CONTINUE

1. Break

L'instruction break permet d'interrompre prématurément une boucle for ou while. Pour illustrer ceci, prenons cet exemple :

```
$compt=1;
while ($compt<5) {
    if ($compt == 4) break;

    echo ("ligne : " + $compt + "<BR>");
```

```
$compt++; }  
echo("sortie forcée de la boucle");
```

Le fonctionnement est semblable à l'exemple précédent sauf que lorsque le compteur vaut 4. A ce moment, par le break, on sort de la boucle et "fin de boucle" est affiché.

Ce qui donnerait à l'écran :

```
ligne      :      1 ligne :      2  
ligne      :      3 sortie forcée de la boucle
```

2. Continue

L'instruction continue permet de sauter une instruction dans une boucle for ou while et de continuer ensuite le bouclage (sans sortir de celui-ci comme le fait break).

```
$compt=1;  
while ($compt<5) {  
    if ($compt == 3){  
        $compt++ continue;}  
    echo ("ligne : " + $compt + "<BR>");  
    $compt++; }  
echo("fin de la boucle");
```

D. L'affichage des données et les inclusions

I. Afficher du texte

La principale fonction de PHP est **echo** qui permet d'imprimer du texte dans votre document. En réalité ce n'est pas vraiment une fonction mais une structure de langage.

Il est très important de mettre un caractère d'échappement (\) devant tous les caractères spéciaux et délimiteurs afin de ne pas créer de **Parse Error** de la part du compilateur PHP. Il confondra évidemment le dit caractère de la chaîne avec celui de la fonction, erreur souvent commise par les programmeurs. Exemples

Voici quelques exemples d'utilisation de PHP. Comme vous pourrez le voir, il faut insérer votre texte entre quotes simples ou doubles à la suite de l'instruction echo.

- Afficher un texte classique

```
<?php echo 'Hello World!'; ?>  
<?php  
echo 'L\'adresse de ce site est www.generationphp.net.';  
echo "L'adresse de ce site est www.generationphp.net.";  
?>
```

- Afficher du HTML

Il vous est permis aussi d'afficher du code HTML.

```
<?php echo ''; ?>
```

- Afficher du JavaScript

Il vous est permis aussi d'afficher du code JavaScript.

```
<?php
```

```
    echo '<script>alert("Aucun enregistrement"); </script>';
```

```
?>
```

- Afficher une variable

La fonction echo permet aussi d'afficher le contenu d'une variable.

```
<?php
```

```
    $variable = 'Hello World!';
```

```
    echo $variable;
```

```
?>
```

- Afficher le résultat d'une fonction

Vous pouvez aussi afficher le résultat d'une fonction.

```
<?php
```

```
    // Afficher une date formatée
```

```
    echo date('Y-m-d');
```

```
?>
```

II. Les Caractères spéciaux

Voici la liste des caractères réservés qui, précédés du signe "\" dans une chaîne de caractères, ont une signification particulière :

Séquence	Valeur
\n	Nouvelle ligne
\r	Retour à la ligne
\t	Tabulation horizontale
\\	Antislash
\\$	Caractère \$
\"	Guillemet double
\[0-7]{1,3}	Une séquence de caractère qui permet d'afficher un nombre en notation octale.
\[0-9A-Fa-f]{1,2}	Une séquence de caractère qui permet d'afficher un nombre en notation hexadécimale.

III. Les inclusions

On peut inclure dans un script php le contenu d'un autre fichier avec require ou include:

1. require :

Insère dans le code le contenu du fichier spécifié même si ce n'est pas du code php. C'est équivalent au préprocesseur *#include* du C.

Exemple :

```
require("fichier.php");
```

2. include:

Evalue et insère à chaque appel (même dans une boucle) le contenu du fichier passé en argument. *Exemple :*

```
include("fichier.php");
```

Partie 2
Les types de données structurés en PHP :
Les tableaux et les chaînes de caractères

Plan

- Les tableaux en PHP
- Les chaînes de caractères
- Les expressions régulières

I. Les tableaux en PHP :

1. Définition :

Un tableau est composé d'un certain nombre d'éléments possédant chacun une valeur propre ainsi qu'une clé ou indice, permettant de faire référence à cet élément. Par défaut, l'indice du premier élément d'un tableau est zéro. Contrairement à des langages tels que le langage C, il est possible de stocker des éléments de types différents dans un même tableau. L'élément d'un tableau peut être un autre tableau.

En PHP, deux types de tableaux sont à utiliser :

- Les tableaux numérotés (à indice de type entier)
- Les tableaux associatifs : des tableaux utilisant des chaînes de caractères au lieu des simples entiers pour définir les indices d'un tableau

2. Les tableaux numérotés

Les tableaux numérotés permettent de stocker une série d'éléments du même type, comme par exemple des prénoms. Chaque élément du tableau contiendra alors un prénom. Un tableau numéroté commence toujours à la case n°0.

a. Initialisation d'un tableau numéroté

Il y a plusieurs façons d'insérer des éléments dans un tableau. On peut tout simplement leur affecter une valeur, de la même façon que pour une autre variable.

Soit l'exemple de tableau suivant, contenu de la variable \$prenoms :

Clé	Valeur
0	Ahmed
1	Aya
2	Aymen
3	Anas
4	Bedis
...	...

- Le tableau numéroté peut être initialisé manuellement case par case de la façon suivante: **Code : PHP**

```
<?php
    $prenoms[0] = 'Ahmed';
    $prenoms[1] = 'Aya';
    $prenoms[2] = 'Aymen';
    .....
?>
```

- Le même code aurait pu être écrit sans spécifier des indices explicites (avec des crochets vides). Comme aucun indice n'a été spécifié de façon explicite entre les crochets [], les éléments possèdent automatiquement des indices respectifs par défaut 0, 1 et 2.

Code : PHP

```
<?php
    $prenoms[] = 'Ahmed'; // Créera $prenoms[0]
    $prenoms[] = 'Aya'; // Créera $prenoms[1]
    $prenoms[] = 'Aymen'; // Créera $prenoms[2]
    .....
?>
```

En pratique pour initialiser un tableau numéroté en PHP, on peut utiliser aussi la fonction array.

Code : PHP

```
<?php
    // La fonction array permet de créer un array
    $prenoms = array ('Ahmed', 'Aya', 'Aymen', 'Anas', 'Bedis');
```

?>

L'ordre a beaucoup d'importance. Le premier élément ("Ahmed") aura le n°0, ensuite Aya le n°1,etc.

b. Afficher un tableau numéroté

Pour afficher un élément, il faut donner sa position entre crochets après \$prenoms.

Exemple : pour afficher "Aya", on doit écrire :

```
<?php
```

```
    echo $prenoms[1];
```

```
?>
```

3. Les tableaux associatifs

Les tableaux associatifs permettent de découper une donnée en plusieurs sous-éléments. Par exemple, une coordonnée peut être découpée en nom, prénom, nom de rue, ville...

Les tableaux associatifs fonctionnent sur le même principe, sauf qu'au lieu de numéroté les cases, on va les étiqueter en leur donnant à chacune un nom différent.

Par exemple, supposons que dans un seul array, nous voulons enregistrer les coordonnées de quelqu'un (nom, prénom, adresse, ville etc...). Si l'array est numéroté, comment savoir que le n°0 est le nom, le n°1 le prénom, le n°2 l'adresse ?... C'est là que deviennent utiles les tableaux associatifs.

a. initialiser un tableau associatif

Pour initialiser un tableau associatif, on utilisera la fonction array , mais avec une "étiquette" devant chaque information :

Code : PHP

```
<?php
```

```
    // On crée notre array $coordonnees
```

```
    $coordonnees = array (
```

```
        'prenom' => 'Aymen',
```

```
        'nom' => 'Gharbi',
```

```
        'adresse' => '3 Rue sassi bellil',
```

```
        'ville' => 'Ariana');
```

```
?>
```

Vous remarquez qu'on peut utiliser une flèche (=>) pour dire "associé à" afin de spécifier les indices particuliers des éléments d'un tableau.

Nous avons créé un tableau qui ressemble à la structure suivante :

Clé	Valeur
prenom	Aymen
nom	Gharbi
adresse	3 Rue sassi bellil
ville	Ariana

Il est aussi possible de créer le tableau case par case comme ceci :

Code PHP

```
<?php
    $coordonnees['prenom'] = 'Aymen';
    $coordonnees['nom'] = 'Gharbi';
    $coordonnees['adresse'] = '3 Rue Sassi bellil';
    $coordonnees['ville'] = 'Ariana';
?>
```

b. Afficher un tableau associatif

Pour afficher un élément, il suffit d'indiquer le nom de cet élément entre crochets, ainsi qu'entre guillemets ou apostrophes puisque l'étiquette du tableau associatif est un texte.

Par exemple, pour extraire la ville, on devra taper :

Code : PHP

```
<?php
    echo $coordonnees['ville'];
?>
```

Autre Exemple :

```
<?php
    $tab2 = array('couleur' => 'rouge', 'gout' => 'sucre', 'forme' => 'rond', 'nom' =>
        'pomme');
    echo ($tab2['gout']); // affiche sucre
    echo ($tab2[0]); // faux!
?>
```

4. Parcourir un tableau

On peut procéder de trois moyens différents pour explorer un tableau selon qu'il est numéroté ou associatif.

Soit :

- La boucle for
- La boucle foreach
- La fonction print_r (utilisée principalement pour le débogage)

a. La boucle for

Pour parcourir un tableau numéroté avec une boucle for il faut incrémenter un compteur à partir de 0 :

```
<?php
    // On crée notre array $prenoms
    $prenoms = array ('Ahmed', 'Aya', 'Aymen', 'Anas', 'Bedis');
    // Puis on fait une boucle pour tout afficher :
```

```

        for ($indice = 0; $indice < 5; $indice++)
            { echo $prenoms[$indice] . '<br />'; // affichera $prenoms[0], $prenoms[1] etc... }
    ?>

```

b. La boucle foreach

La boucle foreach est plus adaptée aux tableaux. C'est une sorte de boucle for spécialisée dans les tableaux. foreach va passer en revue chaque ligne du tableau, et lors de chaque passage, il va mettre la valeur de cette ligne dans une variable temporaire (par exemple \$element).

```

<?php
    $prenoms = array ('Ahmed', 'Aya', 'Aymen', 'Anas', 'Bedis');
    foreach($prenoms as $element) {
        echo $element . '<br />'; // affichera $prenoms[0], $prenoms[1] etc...
    }
?>

```

A chaque tour de boucle, la valeur de l'élément suivant est mise dans la variable \$element. On peut donc utiliser \$element uniquement à l'intérieur de la boucle afin d'afficher l'élément en cours.

L'avantage de foreach est qu'il permet aussi de parcourir les tableaux associatifs.

```

<?php
    $coordonnees = array (
        'prenom' => 'Aymen',
        'nom' => 'Gharbi',
        'adresse' => '3 Rue sassi bellil',
        'ville' => 'Ariana');
    foreach($coordonnees as $element) {
        echo $element . '<br />';
    }
?>

```

foreach va mettre tour à tour dans la variable \$element le prénom, le nom, l'adresse et la ville contenus dans l'array \$coordonnees.

On met donc entre parenthèses :

1. D'abord le nom de l'array (ici \$coordonnees)
2. Ensuite le mot-clé as (qui signifie quelque chose comme "en tant que")
3. Enfin le nom d'une variable que vous choisirez qui va contenir tour à tour chacun des éléments de l'array (ici \$element).

Entre les accolades, on n'utilise donc que la variable \$element. La boucle s'arrête lorsqu'on a parcouru tous les éléments de l'array.

Toutefois, avec cet exemple on ne récupère que la valeur. Or, on peut aussi récupérer la clé de l'élément. On doit dans ce cas écrire foreach comme ceci :

```
<?php foreach($coordonnees as $cle => $element) { } ?>
```

A chaque tour de boucle, on disposera non pas d'une mais de deux variables :

- \$cle : elle contiendra la clé de l'élément en cours d'analyse ("prenom", "nom", etc.).
- \$element : il contiendra la valeur de l'élément en cours ("François", "Dupont", etc.).

Exemple :

```
<?php
$coordonnees = array (
    'prenom' => 'Aymen',
    'nom' => 'Gharbi',
    'adresse' => '3 Rue sassi bellil',
    'ville' => 'Ariana');
foreach ($coordonnees as $cle => $element) {
    echo '[' . $cle . '] vaut ' . $element . '<br />';
}
?>
```

c. Afficher rapidement un tableau avec print_r

C'est une fonction qui permet juste de savoir ce que contient l'array. C'est une sorte de echo spécialisé dans les array.

Remarque : Cette commande ne renvoie pas de code HTML comme
 pour les retours à la ligne. Pour bien voir les retours à la ligne, il faut donc utiliser la balise HTML <pre> qui nous permet d'avoir un affichage plus correct.

```
<?php
$coordonnees = array (
    'prenom' => 'Aymen',
    'nom' => 'Gharbi',
    'adresse' => '3 Rue sassi bellil',
    'ville' => 'Ariana');
echo '<pre>';
print_r($coordonnees);
echo '</pre>';
```

5. Recherche dans un tableau

On peut avoir trois types de recherches, basées sur des fonctions PHP :

- array_key_exists : pour vérifier si une clé existe dans le tableau
- in_array : pour vérifier si une valeur existe dans le tableau
- array_search : pour récupérer la clé d'une valeur dans le tableau

a. *Vérifier si une clé existe dans le tableau : array_key_exists*

Voici notre problème : on a un tableau, mais on ne sait pas si la clé qu'on cherche est dedans. On va utiliser pour vérifier ça la fonction `array_key_exists` qui va parcourir le tableau pour nous et nous dire si le tableau contient cette clé. On doit lui donner d'abord le nom de la clé à rechercher, puis le nom de l'array dans lequel on fait la recherche :

```
<?php array_key_exists('cle', $array); ?>
```

La fonction renvoie un booléen, c'est à dire true (vrai) si la clé est dans l'array, et false (faux) si la clé ne se trouve pas dans l'array. Ça nous permet de faire un test facilement avec un if :

```
<?php
$coordonnees = array (
    'prenom' => 'Aymen',
    'nom' => 'Gharbi',
    'adresse' => '3 Rue sassi bellil',
    'ville' => 'Ariana');
if (array_key_exists('nom', $coordonnees))
    { echo 'La clé "nom" se trouve dans les coordonnées !'; }

if (array_key_exists('pays', $coordonnees))
    { echo 'La clé "pays" se trouve dans les coordonnées !'; }
?>
```

b. *Vérifier si une valeur existe dans l'array : in_array*

Le principe est le même que `array_key_exists`... mais cette fois on recherche dans les valeurs. `in_array` renvoie true si la valeur se trouve dans l'array, false si elle ne s'y trouve pas.

```
<?php
$fruits = array ('Banane', 'Pomme', 'Poire', 'Cerise', 'Fraise', 'Framboise');
if (in_array('Myrtille', $fruits))
    { echo 'La valeur "Myrtille" se trouve dans les fruits !'; }
if (in_array('Cerise', $fruits))
    { echo 'La valeur "Cerise" se trouve dans les fruits !'; }
?>
```

On ne voit que le message pour la Cerise, tout simplement parce que `in_array` a renvoyé true pour "Cerise" et false pour "Myrtille".

c. *Récupérer la clé d'une valeur dans le tableau : array_search*

`array_search` fonctionne comme `in_array` : il travaille sur les valeurs d'un tableau. Donc la fonction renvoie :

- Si elle a trouvé la valeur, `array_search` renvoie la clé correspondante (c'est-à-dire le numéro si c'est un array numéroté, ou le nom de la clé si c'est un array associatif).
- Si elle n'a pas trouvé la valeur, `array_search` renvoie `false`.

<?php

```
$fruits = array ('Banane', 'Pomme', 'Poire', 'Cerise', 'Fraise', 'Framboise');
$position = array_search('Fraise', $fruits);
echo "'Fraise' se trouve en position " . $position . "<br />";
$position = array_search('Banane', $fruits);
echo "'Banane' se trouve en position " . $position;
```

?>

6. Trier un tableau :

PHP dispose de plusieurs fonctions permettant de trier un tableau.

La plupart de ces fonctions ne retournent pas de valeur mais modifient simplement le tableau passé en paramètre.

Les fonctions **`sort()`** et **`rsort()`** permettent de trier un tableau par valeur croissante ou décroissante. Les indices sont changés.

Exemple :

```
<? php
    $pays = array ("us" => "Etats-Unis",
                  "ch" => "Suisse",
                  "ca" => "Canada",
                  "fr" => "France",
                  "de" => "Allemagne");

    sort ($pays);
    foreach($pays as $cle => $element) { echo "L'élément $cle est égal à $val<BR>\n"; }
?>
```

Cet exemple imprime :

```
L'élément 0 est égal à Canada
L'élément 1 est égal à Suisse
L'élément 2 est égal à Allemagne
L'élément 3 est égal à France
L'élément 4 est égal à Etats-Unis
```

Cela réorganise le tableau afin que les valeurs soient en ordre alphabétique et que les indices reflètent également cet ordre.

On peut également trier un tableau par indice, à l'aide des fonctions **`ksort()`** et **`krsort()`**, qui s'utilisent de la même manière que les fonctions précédentes.

Exemple :

<? php

```
$pays = array ("e" => "Etats-Unis" ,
               "d" => "Suisse",
               "c" => "Canada",
               "b" => "France",
               "a" => "Allemagne");
ksort ($pays);
foreach($pays as $cle=>$element){ echo "L'élément $cle est égal à; $val<BR>\n"; }

?>
```

Cet exemple imprime :

L'élément a est égal à Allemagne
 L'élément b est égal à France
 L'élément c est égal à Canada
 L'élément d est égal à Suisse
 L'élément e est égal à Etats-Unis

La fonction **usort()** est un peu plus complexe. Elle prend un tableau comme argument, comme toutes les autres fonctions de tri, mais accepte également un second argument.

Il s'agit d'une fonction définie par l'utilisateur et indiquant à usort() comment effectuer le tri. L'exemple ci-dessous trie le tableau selon la longueur de la chaîne contenue dans ses éléments.

Exemple :

```
<? php

function par_taille ($a, $b) {
    $_a = strlen ($a);  $_b =
    strlen ($b);  if ($_a == $_b)
    return 0;  return ($_a <
    $_b) ? -1 : 1;
}
$pays = array ("e" => "Etats-Unis" ,
               "d" => "Suisse",
               "c" => "Canada",
               "b" => "France",
               "a" => "Allemagne");
usort ($pays, par_taille);
foreach($pays as $cle => $element { echo "L'élément $cle est égal à $val<BR>\n"; }

?>
```

Ce code imprime :

L'élément 0 est égal à Suisse
 L'élément 1 est égal à Canada
 L'élément 2 est égal à France
 L'élément 3 est égal à Allemagne

L'élément 4 est égal à Etats-Unis

La fonction de comparaison doit retourner un entier, qui sera inférieur, égal ou supérieur à zéro suivant que le premier argument est considéré comme plus petit, égal ou plus grand que le second argument. Si les deux arguments sont égaux, leur ordre est indéfini

II. Les Chaînes de caractères :

1. Définition:

Une chaîne de caractères est un ensemble de caractères entre guillemets simples ou doubles. Exemple: `$Variable = "Bonjour tout le monde";`

Il existe des caractères repérés par un code ASCII spécial permettant d'effectuer des opérations particulières. Ces caractères peuvent être représentés plus simplement en langage PHP grâce au caractère `\` suivi d'une lettre, qui précise qu'il s'agit d'un caractère de contrôle:

Caractère	Description
<code>\"</code>	Guillemet
<code>\\</code>	Barre oblique inverse (antislash)
<code>\r</code>	retour chariot
<code>\n</code>	retour à la ligne
<code>\t</code>	Tabulation

En effet, certains de ces caractères ne pourraient pas être représentés autrement (un retour à la ligne ne peut pas être représenté à l'écran). D'autre part, les caractères `\` et `"` ne peuvent pas faire partie en tant que tel d'une chaîne de caractère.

Remarque :

Les chaînes placées entre doubles guillemets sont sujettes aux substitutions de variables et au traitement à l'aide de séquences d'échappement, tandis que les chaînes entre guillemets simples ne le sont pas.

Exemple :

```
echo (" bienvenue \n à notre site ") ; // donne bienvenue retour à la ligne à notre site
echo (' bienvenue \n à notre site ') ; // donne bienvenue \n à notre site
```

2. Fonctions de chaînes de caractères :

Nous allons détailler ci-dessous les fonctions de traitement de chaînes de caractères les plus importantes de PHP.

- **strlen()** :

int **strlen** (string str)

Retourne la longueur de la chaîne string.

Exemple :

```
if(strlen($userinput) >30) echo("La saisie ne doit pas dépasser 30 caractères");
```

- **substr()** :

string **substr**(string source, int début, int [taille])

Retourne une portion de **string**, spécifiée avec le début **début** et la longueur **taille**.

Exemple :

```
echo(substr("Christophe"),-2); Imprime: he
```

```
echo(substr("Christophe"),-5,3); Imprime : top
```

=> Renvoie une chaîne de trois caractères débutant au cinquième caractère en partant de la fin.

```
echo(substr("Christophe"),-6,-3); Imprime : sto
```

=> Du sixième avant la fin au troisième avant la fin.

Lorsqu'une taille négative est spécifiée, la chaîne renvoyée se terminera à cette distance de la fin de la chaîne source.

- **trim()** :

string **trim** (string **str**)

Cette fonction retire les espaces blancs de début et de fin de chaîne, et retourne la chaîne nettoyée. Les espaces blancs sont : "\n", "\r", "\t", "\v", "\0", et " " (espace).

Exemple :

```
echo(trim(" chaîne exemple ")); //imprime 'chaîne vide'
```

- **implode()**

string **implode** (string *separator*, array *tableau*)

Retourne une chaîne constituée de tous les éléments du tableau, pris dans l'ordre, transformés en chaîne, et séparés par séparateur.

- **explode()**

array **explode** (string *separator*, string *string*)

Retourne un tableau qui contient les éléments de la chaîne string, séparés par separator.

Exemple :

```
<?php
$chaîne="Nom|Prenom|Adresse";
$champs=explode("|",$chaîne);
$boucle=0;
while($boucle<sizeof($champs)
){ echo$champs[$boucle];
$boucle++;
}??>
```

- ***str_replace()***

string ***str_replace***(modèle, remplacement, chaîne)

Remplace toutes les occurrences de modèle dans chaîne par remplacement

Exemple :

```
<?php
$chaîne="tout est rouge";
$chaîne=str_replace("rouge", "bleu", $chaîne);
echo $chaîne;
?>
```

Imprime : "tout est bleu"

Autres fonctions:

- ***strtolower(\$str)*** : conversion en minuscules
- ***strtoupper(\$str)*** : conversion en majuscules
- ***strcmp(\$str1,\$str2)*** : comparaison de 2 chaînes
- ***addslashes(\$str)*** : déspecialise les caractères spéciaux (', ", \)
- ***ord(\$char)*** : retourne la valeur ASCII du caractère ***\$char***

III. Les expressions régulières

1. Définition:

Une expression régulière permet de caractériser le format d'une chaîne de caractère. Plus précisément, une expression régulière est un motif ou une série de motifs de caractères. Le but étant de rechercher ou de remplacer un motif dans une chaîne de caractère.

2. Syntaxe d'une expression régulière :

Une expression régulière est une chaîne de caractères contenant des caractères spéciaux et des caractères standards. Il existe une grande variété de caractères spéciaux, le plus facile est de les voir au fur et à mesure à l'aide d'exemples.

- Les symboles ***^*** et ***\$*** indiquent le début ou la fin d'une chaîne, afin de la délimiter

Exemples :

^debut: Chaîne qui commence par *"debut"*

fin\$: Chaîne qui se termine par *"fin"*

^chaîne\$: Chaîne qui commence et se termine par *"chaîne"*

abc: chaîne contenant la chaîne *"abc"*

Les symboles *******, ***+*** et ***?***, respectivement *"zero ou plusieurs"*, *"un ou plusieurs"*, *"un ou aucun"*, permettent de donner une notion de nombre.

Exemples :

abc+: chaîne qui contient *"ab"* suivie de un ou plusieurs *"c"*

("abc", "abcc" etc..)

"abc*": chaîne qui contient "ab" suivie de zéro ou plusieurs "c" ("ab", "abc" etc..)

"abc?": chaîne qui contient "ab" suivie de zéro ou un "c" ("ab" ou "abc")

"^abc+": chaîne qui commence par "ab" suivie de un ou plusieurs "c"
("abc", "abcc" etc..)

☞ Les accolades {X,Y} permettent de donner des limites de nombre.

Exemples :

"abc{2}": chaîne qui contient "ab" suivie de deux "c" ("abcc")

"abc{2,}": chaîne qui contient "ab" suivie de deux "c" ou plus
("abcc" etc..)

"abc{2,4}": chaîne qui contient "ab" suivie 2, 3 ou 4 "c" ("abcc" ..
"abcccc")

A noter que le premier nombre de la limite ("{0,2}", et pas "{,2}") est obligatoire. Les symboles vus précédemment ('*', '+', and '?') sont équivalents à "{0,}", "{1,}", et "{0,1}".

- Les parenthèses () permettent de représenter une séquence de caractères.

Exemple :

"a(bc)*": chaîne qui contient "a" suivie de zéro "bc" ou plus

- La barre verticale | se comporte en tant qu'opérateur OU.

Exemple :

"un|le": chaîne qui contient "un" ou "le"

"(un|le) chien": chaîne qui contient "un chien" ou "le chien"

"(a|b)*": chaîne qui contient une suite de "a" ou de "b"

- Le point . indique n'importe quel caractère (une fois)

Exemple :

"^.{3}\$": chaîne qui contient 3 caractères

- Les crochets [] définissent une liste de caractères autorisés (ou interdits). Le signe - permet, quant à lui, de définir un intervalle. Le caractère ^ après le premier crochet indique une interdiction.

Exemples :

"[abc]": chaîne qui contient un "a", un "b", ou un "c"

"[a-z]": chaîne qui contient un caractère compris entre "a" et "z"

"[^a-zA-Z]": chaîne qui ne commence **pas** par une lettre

Pour rechercher un caractère faisant partie des caractères spéciaux, il suffit de le faire précéder d'un antislash (un antislash doit donc être doublé), SAUF entre crochets. En effet, dans les crochets, chaque caractère représente ce qu'il est. Pour représenter un] il faut le mettre en premier (ou après un ^ si c'est une interdiction), un - se met en premier ou en dernier.

Exemples :

"[\+?{}.]": chaîne qui contient un de ces six caractères

"[]-": chaîne qui contient le caractère "]" ou le caractère "-"

Partie 3: Les fonctions en PHP

Plan

- Les fonctions
- Les formulaires

I. Principe et syntaxe de base

Une fonction est un bloc de code défini une fois, puis invoqué à partir d'autres endroits du programme.

Une fonction peut accepter un ou plusieurs arguments, accomplit un ensemble d'opérations prédéfinis selon ces arguments puis renvoie une valeur en résultat. Une fonction peut être définie en utilisant la syntaxe suivante :

```
function ma_fonction ($arg_1, $arg_2, ..., $arg_n)  
{  
    //Corps de ma fonction...  
return $valeur_de_retour ;  
}
```

En PHP toute fonction doit être préalablement définie avant d'être utilisée. Les valeurs sont renvoyées en utilisant une instruction de retour optionnelle. Tous les types de variables peuvent être renvoyés (tableaux et objets compris).

Exemple :

```
function carre($nombre)
{
return $nombre * $nombre ;
}
echo carre(3) ; # affiche la valeur 9
```

II. Les arguments de fonction :

Les arguments constituent un moyen de transmettre des données à la fonction. Par défaut les arguments sont transmis par **valeur** : Cela signifie que la variable paramètre interne à la fonction contient une copie de la valeur qui lui a été transmise. Si la valeur du paramètre change, cela ne modifie pas la valeur de la variable. **Exemples :**

```
<?php
//Fonction sans parametre : déclaration
function alert()
{
    echo "Alert ! Alert ! Alert
!"; }

// Appel de la fonction
alert();
```

```
//Fonction avec deux paramètres
```

```
//définition :
```

```
function addition ($a,$b)
{
```

```

        $c=$a+$b;
print ($c);
    }
    //appel :
    addition (2,10);
?>

```

Le résultat affiche 12 .Vous voyez ici qu'il y a passage de variables vers la fonction et que celle-ci récupère ses variables pour les traiter en interne

```

<?php
function imprime_double($n)
{
    $n = $n * 2 ;
    echo($n) ;}
$a = 5 ;
echo ($a) ; //imprime 5
imprime_double($a) ; //imprime 10
?>

```

La valeur de \$a n'est pas modifiée, même après avoir été transmise à imprime_double(), puisque l'argument a été transmis comme valeur.

En revanche, si un argument est transmis par référence, toute modification apportée à la variable paramètre modifie la variable de l'instruction d'appel.

Pour passer un argument par référence, il faut ajouter un '&' devant l'argument dans la déclaration de la fonction.

Exemple :

```

function augmentation(&$salaire, $pourcent) {
    //augmente $salaire de $pourcent
    $salaire+=$salaire * $pourcent/100 ;
}

$sal = 50000 ; echo("Salaire avant augmentation : $sal) ;
//Imprime 50000 augmntation($sal, 4) ;

echo ("Salaire après augmentation : $sal<BR>\n") ; //Imprime 52000 Si vous
souhaitez passer une variable par référence à une fonction mais de manière

```

ponctuelle, vous pouvez ajouter un '&' devant l'argument dans l'appel de la fonction.

Exemple :

```
function augmentation($salaire, $pourcent) {  
    //augmente $salaire de $pourcent  
    $salaire+=$salaire * $pourcent/100 ;  
}  
  
$sal = 50000 ; echo("Salaire avant augmentation : $sal) ;  
//Imprime 50000 augmntation(&$sal, 4) ;  
echo ("Salaire après augmentation : $sal<BR>\n") ; //Imprime 52000
```

III. Portée d'une variable :

La portée d'une variable détermine quelles parties du programme y ont accès.

Pour la plupart des variables, la portée concerne la totalité d'un script PHP. Mais, lorsque vous définissez une fonction, la portée d'une variable définie dans cette fonction est locale à la fonction.

Exemple :

```
$var="a" ;  
function change_var() { $var = "b"  
; } change_var() ; echo("$var") ;  
//affiche "a"
```

Ce code imprime "a". La variable \$var à l'intérieur de la fonction possède une portée locale, et est de ce fait différente de la variable \$var externe à la fonction.

La variable extérieure à la fonction possède une portée globale, car elle peut être atteinte et modifiée par tout fragment du code du script général, hors d'une fonction.

Deux façons existent pour accéder à une variable globale au sein d'une fonction :

- Déclarer la variable comme global au sein de la fonction,
- Utiliser le tableau associatif \$GLOBALS avec comme clé le nom de la variable

globale. **Exemple :**

```
$var="a" ;
```

```
function change_var() { global $var ;      $var = "b" ; }
change_var() ;
echo("$var") ; //affiche "b"
```

Le code imprime maintenant "b", parce que \$var fait référence à la même variable, à l'intérieur et à l'extérieur de la fonction.

On peut également avoir recours au tableau intégré \$GLOBALS, qui contient toutes les variables globales du script : **Exemple :**

```
$var="a" ; function
change_var() {
    $GLOBALS["var"] = "b" ;}
change_var() ;
echo("$var") ; //affiche "b"
```

Remarquez que l'opérateur \$ ne figure pas avant le mot var lors de l'utilisation de ce tableau.

1. Variables statiques :

Une variable statique est une variable locale qui ne perd pas sa valeur à chaque fois que la fonction est exécutée. On utilise, comme en C, l'attribut static pour déclarer une telle variable.

Ce type de variable est très utile pour la création de fonctions récursives.

Exemple :

```
function compte () {
    static $compteur = 0 ;
    $compteur++ ; echo
    "$compteur " ;
    if ($compteur < 10) compte() ;
}
compte() ;
```

Lors du tout premier appel de la fonction compte(), la variable statique \$compteur va être créée et se voir affecter la valeur spécifiée (ici 0), mais par la suite elle conservera sa valeur entre chaque appel de la fonction : La variable ne sera pas réinitialisée.

Remarque : Lorsque vous souhaitez qu'une fonction retourne plusieurs valeurs, le plus simple est d'utiliser un tableau : <?

```
function nom_fonction()  
{.....  
    return array( $variable1, $variable2, $variable3 ); // on retourne les valeurs voulues dans un  
tableau  
}  
$retour = nom_fonction(); echo "$retour[0]  
- $retour[1] - $retour[2]"; ?>
```